

Lista de Linguagens de Programação – 12

Nome: _____ Matrícula: _____

1. Escreva um tipo de dados **suit**, cujos valores sejam os quatro naipes de um baralho: **Hearts**, **Clubs**, **Diamonds** e **Spades**.
2. Usando a definição do exercício anterior, escreva a função **suitname** do tipo **suit -> string** que retorne um valor do tipo **string** descrevendo o nome do naipe.
3. Escreva um tipo de dados **number**, cujos valores sejam ou números inteiros ou números reais.
4. Usando a definição do exercício anterior, escreva uma função **plus** de tipo **number -> number -> number** que some dois números. A sua função deve converter inteiros para reais quando receber um parâmetro real e um parâmetro inteiro.
5. Escreva uma função **addUp** do tipo **intnest -> int** que some todos os inteiros em um **intnest**. Este tipo algébrico é definido abaixo:

```
datatype intnest = INT of int | LIST of intnest list
```

6. Escreva a função `prod` do tipo `int mylist -> int` que receba um `int mylist x` e retorne o produto de todos os elementos de `x`. Se a lista for `NIL`, a sua função deverá retornar o número 1. A definição de `mylist` é dada abaixo:

```
datatype 'element mylist = NIL | CONS of 'element * 'element mylist
```

7. Escreva a função `reverse` do tipo `'a mylist -> 'a mylist` que receba um `mylist a` e retorne um `mylist` de todos os elementos de `a`, em ordem inversa. Use a definição de `mylist` do exercício anterior.

8. Escreva a função `append` do tipo `'a mylist -> 'a mylist -> 'a mylist` que receba dois valores do tipo `mylist`, por exemplo, `a` e `b`, e retorne um valor do tipo `mylist` contendo todos os elementos de `a` seguidos de todos os elementos de `b`. Use a definição de `mylist` do exercício 6.

9. Podemos representar uma árvore binária em SML usando o tipo algébrico abaixo:

```
datatype 'data tree = Empty | Node of 'data tree * 'data * 'data tree
```

Um nodo `Empty` é um sentinela usado para indicar que chegamos ao fim de um caminho na árvore; já um nodo `Node` contém uma sub-árvore à direita, um item de dado, e uma sub-árvore a esquerda. Nesta questão você deve implementar uma função `revTree`, cujo tipo é `'a tree -> 'a tree`. Esta função inverte o conteúdo de uma árvore:

```
-revTree (Node(Node(Empty, 1, Empty), 2, Node(Empty, 3, Empty)));  
val it = Node(Node(Empty, 3, Empty), 2, Node(Empty, 1, Empty)) : int tree
```

10. Escreva a função `appendall` do tipo `'a list tree -> 'a list` que receba uma árvore de listas e retorne a lista resultante da concatenação de todas as listas na árvore. Coloque as listas juntas em uma viagem in-order na árvore, isto é, primeiro todas as listas da parte esquerda de um nodo, então a lista do próprio nodo, e finalmente todas as listas à direita do nodo.

11. Uma árvore binária é chamada *completa* se cada nodo tem, ou dois filhos, ou nenhum filho. Em termos de nosso tipo algébrico `tree`, a árvore é completa se cada nodo tem, ou dois filhos do tipo `Empty`, ou dois filhos do tipo `Node`, mas não um de cada. Escreva uma função `isComplete` do tipo `'a tree -> bool` que teste se um valor `tree` é completo.

12. Uma árvore de busca binária é uma árvore binária com algumas propriedades especiais. Primeiro, ela pode ser vazia (`Empty`). Ela também pode ser um `Node` contendo uma árvore à esquerda, um elemento `x` e uma árvore à direita. Neste caso, todos os dados na árvore precisam ser diferentes. Mais ainda, todos os itens na árvore à esquerda precisam ser menores que `x`, e todos os itens à direita precisam ser maiores que `x`. Obviamente as sub-árvores também precisam ser árvores binárias de busca. Escreva uma função `makeBST`, do tipo `'a list -> ('a * 'a -> bool) -> 'a tree` que organize os itens da lista em uma árvore binária. Sua árvore não precisa ser balanceada. Você pode assumir que a lista de entrada contém somente elementos diferentes.

13. Escreva uma função `searchBST` do tipo `''a tree -> (''a * ''a -> bool) -> ''a -> bool` que busque um elemento em uma árvore binária. É óbvio que um aluno de nossa universidade, depois de várias AEDs e afins não vai procurar o item em todos os nodos da árvore binária, não é?

14. Considere o tipo algébrico `exp` abaixo, que descreve uma linguagem muito simples, com as operações aritméticas de adição, multiplicação e menos unário:

```
datatype exp = NUM of int |  
              SUM of exp * exp |  
              MUL of exp * exp |  
              UNMINUS of exp
```

Implemente uma função `interpret`, de tipo `fn : exp -> int` que interprete esta linguagem. Seu interpretador deve assumir a semântica tradicional das operações aritméticas presentes. Por exemplo, dada a expressão `val x = SUM((MUL (NUM 2, NUM 3)), SUM(UNMINUS (NUM 3), NUM 5))`, que corresponde à expressão aritmética $(2 \times 3) + ((-3) + 5)$, a chamada `interpret x` deve retornar o valor inteiro 8.